



Programación de aplicaciones en Spark

Rafael Pastor Vargas
Andrés Duque Fernández
SCC-ETSI Informática-UNED

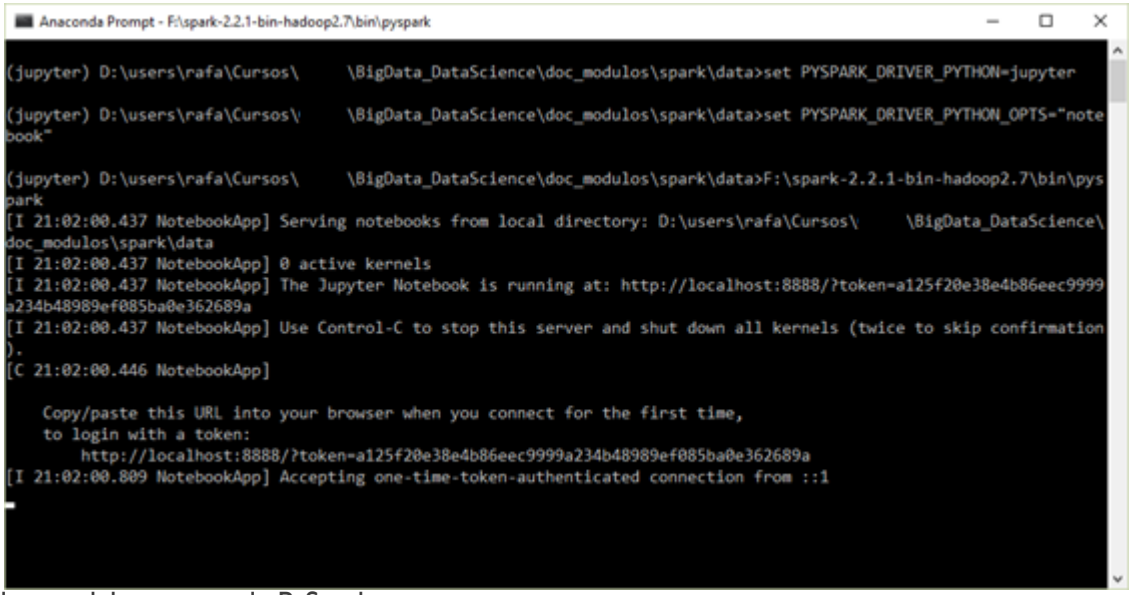
PROGRAMACIÓN DE APLICACIONES EN SPARK

- Entornos de programación de aplicaciones Spark
- Desarrollo y ejecución de una aplicación Spark.
- Creación de sesiones: objetos SparkSession/SparkContext.
- Inicialización de Spark en varios lenguajes: Scala/Python.
- Cargar y guardar datos en Spark.
- Colecciones de datos: Resilient Distributed Dataset, DataFrames y Datasets.
- Paralelización de las colecciones de datos.
- Operaciones sobre RDD.
- El API de Datasets/DataFrames.

ENTORNOS DE DESARROLLO PARA SPARK

- Entornos interactivos
  - Scala (Spark-Shell)
  - Python (Pyspark)
    - **Jupyter Notebooks**
  - R (SparkR)
- Entornos de compilación de trabajos
  - Scala/Java (**Eclipse**/IntelliJ)
  - Python

APLICACIONES/JOBS BASADAS EN JUPYTER NOTEBOOKS Y SPARK (1/3)



- Ejecutar Jupyter dentro del contexto de PySpark
- Linux

  export PYSPARK_DRIVER_PYTHON=jupyter

  export PYSPARK_DRIVER_PYTHON_OPTS='notebook'
- Windows

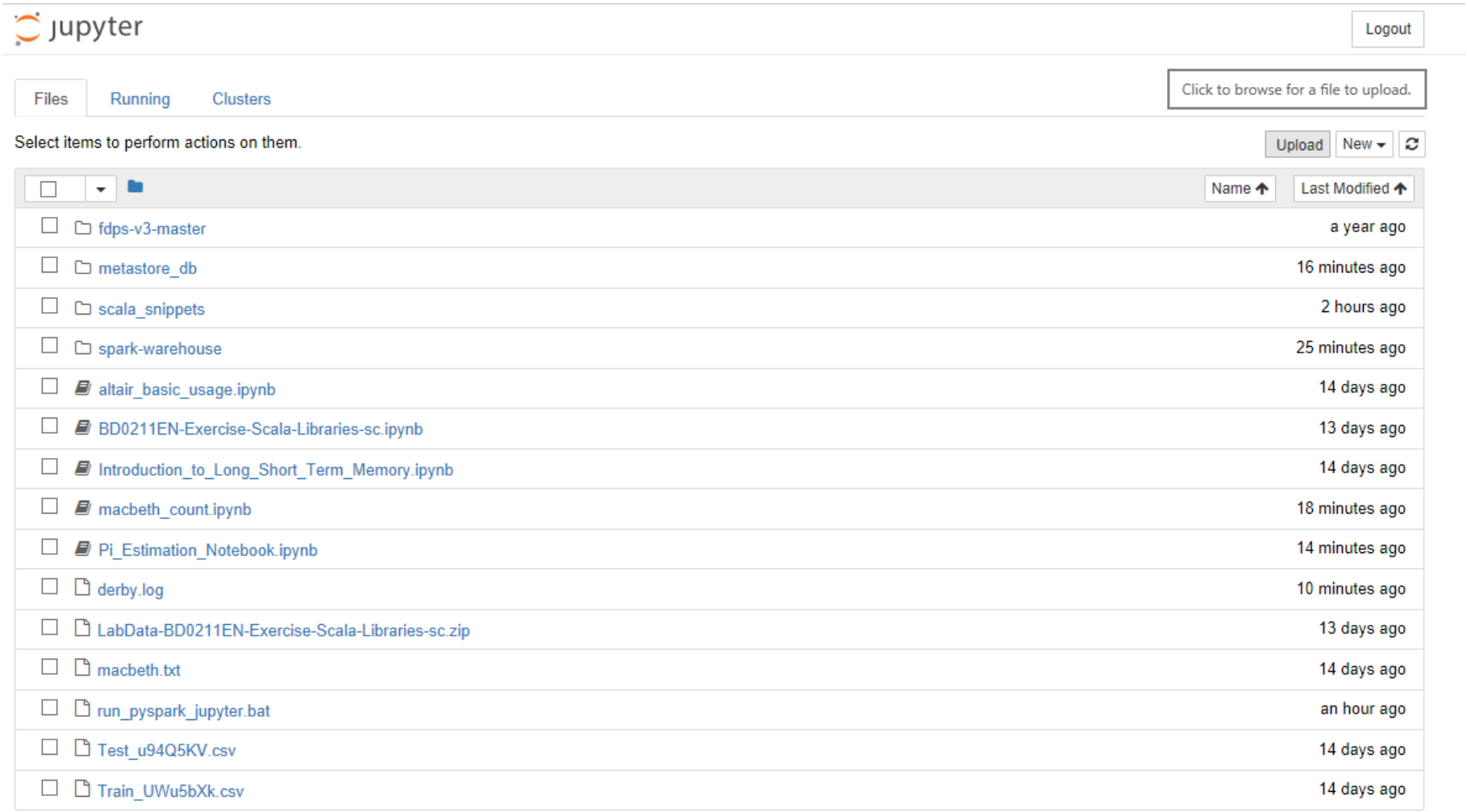
  set PYSPARK_DRIVER_PYTHON=jupyter

  set PYSPARK_DRIVER_PYTHON_OPTS="notebook“

<SPARK_HOME>\bin\pyspark

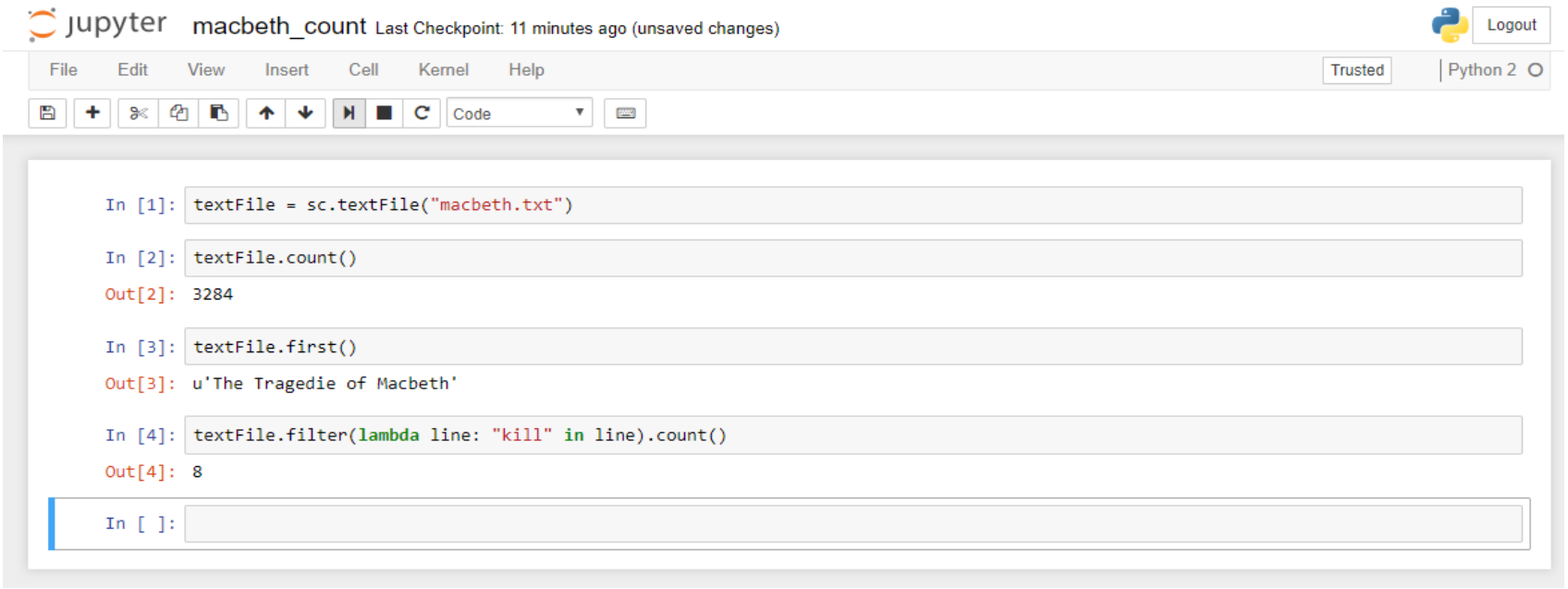
- Es conveniente ejecutarlo desde el directorio donde estén ubicados los notebooks/datos

APLICACIONES/JOBS BASADAS EN JUPYTER NOTEBOOKS Y SPARK (2/3)



APLICACIONES/JOBS BASADAS EN JUPYTER NOTEBOOKS Y SPARK (3/3)

- Visualizar del fichero Macbeth.txt el número de líneas, la primera de ellas y cuantas veces aparece la palabra "kill"



SPARKSESSION Y SPARKCONTEXT (1/4)

- Antes de Spark 2.x: SparkContext, SqlContext, HiveContext
  - SparkContext representaba la conexión con el entorno de ejecución de Spark
- Spark 2.x introduce el API de alto nivel para Datasets / Dataframes
  - SparkSession pasa a ser la nueva representación de la conexión
    - org.apache.spark.sql.SparkSession
    - pyspark.sql.sparkSession
  - El objeto SparkSession encapsula ahora a SparkContext como una propiedad
- Para crear/leer/escribir Datasets/Dataframes → SparkSession
- Para crear/leer/escribir RDD → SparkContext

SPARKSESSION Y SPARKCONTEXT (2/4)

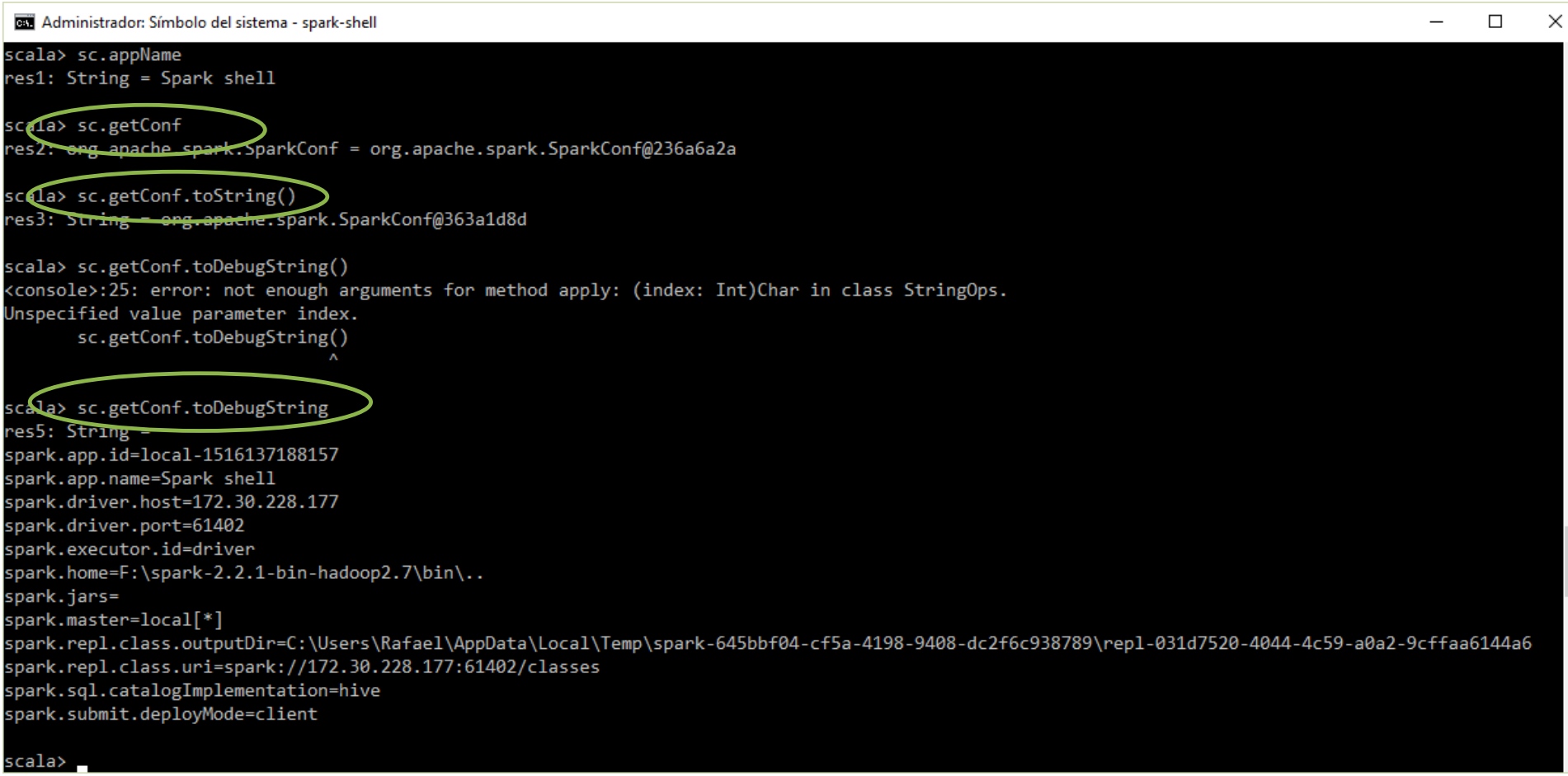
- Spark-Shell y Pyspark crean los objetos automáticamente

```
val sparkSession = new
SparkSession.builder.master(master_path).appName("application
name").config("optional configuration parameters").getOrCreate()
```

- Propiedades/métodos importantes del objeto
  - appName
  - getConf
  - getExecutorMemoryStatus
  - Master
  - Version



SPARKSESSION Y SPARKCONTEXT (3/4)



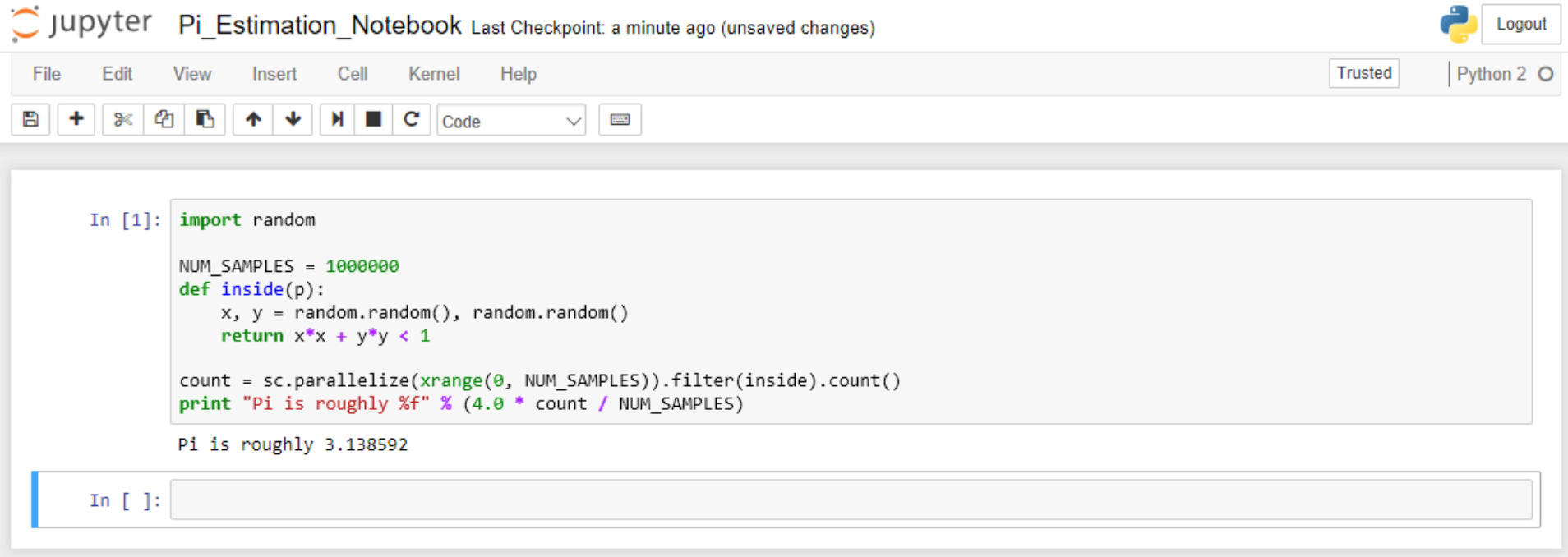
SPARKSESSION Y SPARKCONTEXT (4/4)

- Se comparten APIs en las versiones Scala y Phyton, pero hay algunas funciones/propiedades que no están en Python (getExecutorMemoryStatus)

Method	Use
`addJar(path)`	This method adds the JAR file for all the future jobs that would run through the `SparkContext` object.
`addFile(path)`	This method downloads the file to all the nodes on the cluster.
`listFiles/listJars`	This method shows the list of all the currently added files/JARs.
`stop()`	This method shuts down `SparkContext`.
`clearFiles()`	This method removes the files so that new nodes will not download them.
`clearJars()`	This method removes the JARs from being required for future jobs.

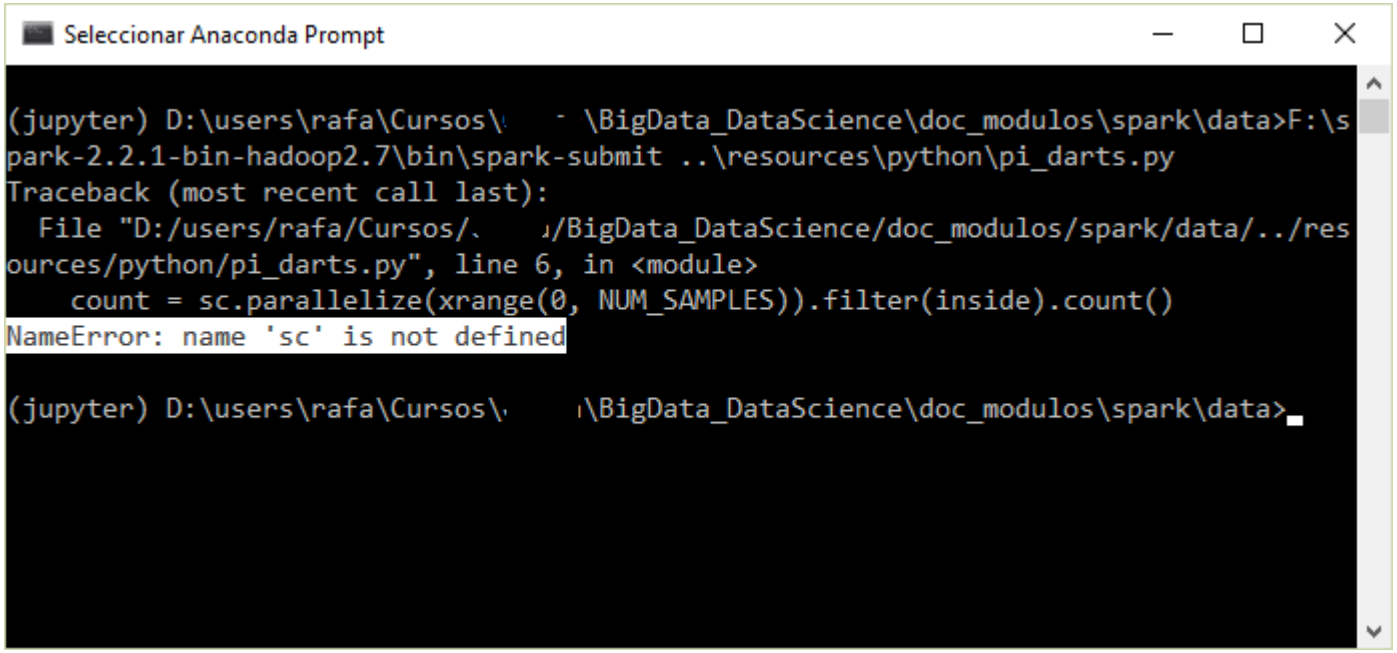
OBJETOS SESSION EN JOBS PYTHON (1/4)

- Código funcionando en un notebook



OBJETOS SESSION EN JOBS PYTHON (2/4)

- Ejecutar la aplicación en spark
  - spark-submit <ubicación fichero py>



OBJETOS SESSION EN JOBS PYTHON (3/4)

- En un notebook (pyspark), los objetos SparkSession (spark) y SparkContext (sc) se crean automáticamente.
- La aplicación debe crearlos explícitamente dentro del código
  - Añadir

```
from pyspark.sql import SparkSession
```

  - Crear la sesión Spark y obtener el contexto spark

```
spark = SparkSession.builder.appName("Pi estimation, python Job").getOrCreate()
```

```
sc = spark.sparkContext
```

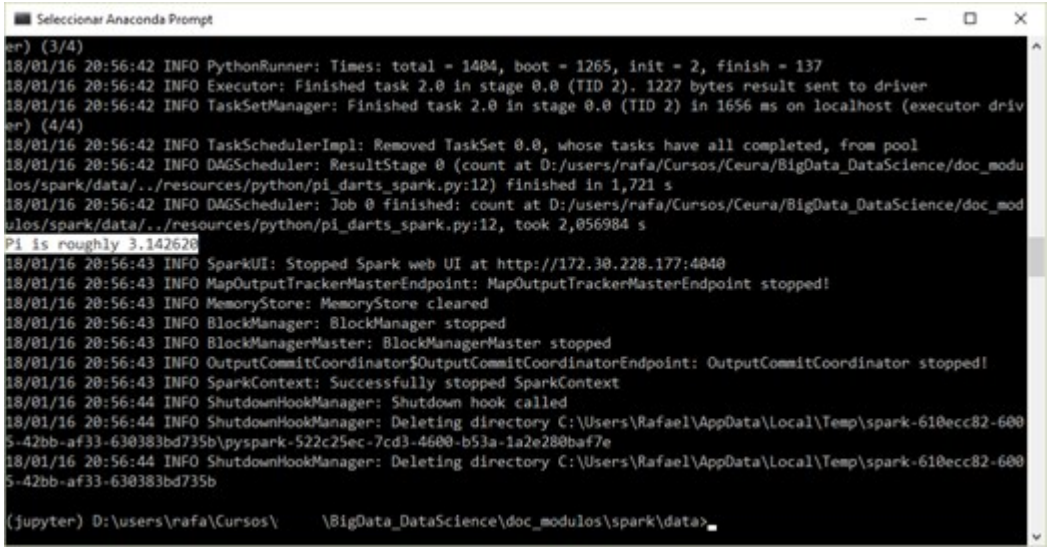
  - Cerrar la sesión al final

```
spark.stop()
```

OBJETOS SESSION EN JOBS PYTHON (4/4)

```
from pyspark.sql import SparkSession
import random

NUM_SAMPLES = 1000000
def inside(p):
    x, y = random.random(), random.random()
    return x*x + y*y < 1
```




```
spark = SparkSession.builder.appName("Pi estimation, python Job").getOrCreate()
sc = spark.sparkContext

count = sc.parallelize(range(0, NUM_SAMPLES)).filter(inside).count()
print ("Pi is roughly %f" % (4.0 * count / NUM_SAMPLES))

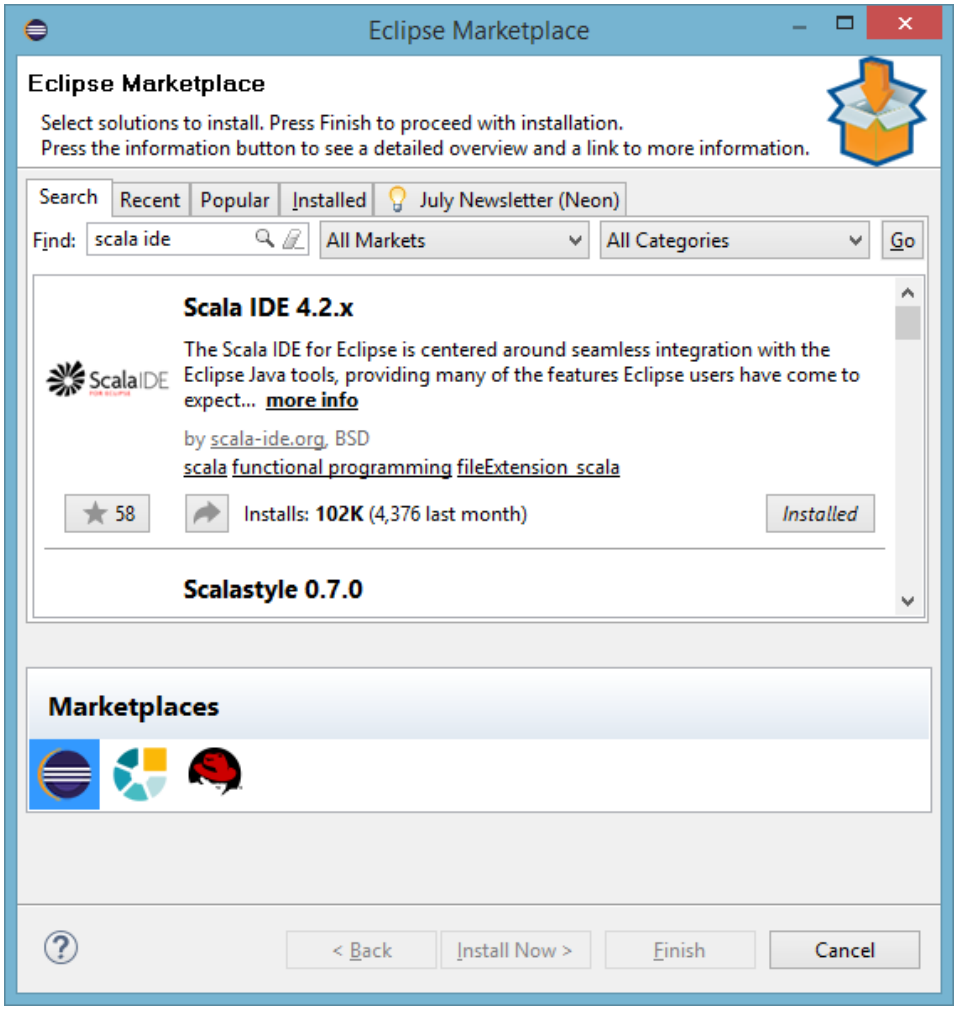
spark.stop()
```

APLICACIONES/JOBS EN SCALA/JAVA (1/9)

- IDE: Eclipse, IntelliJ
- Zeppelin puede ser una alternativa para notebooks con soporte de Scala/Python/SQL
- Maven permite la compilación/despliegue de aplicaciones

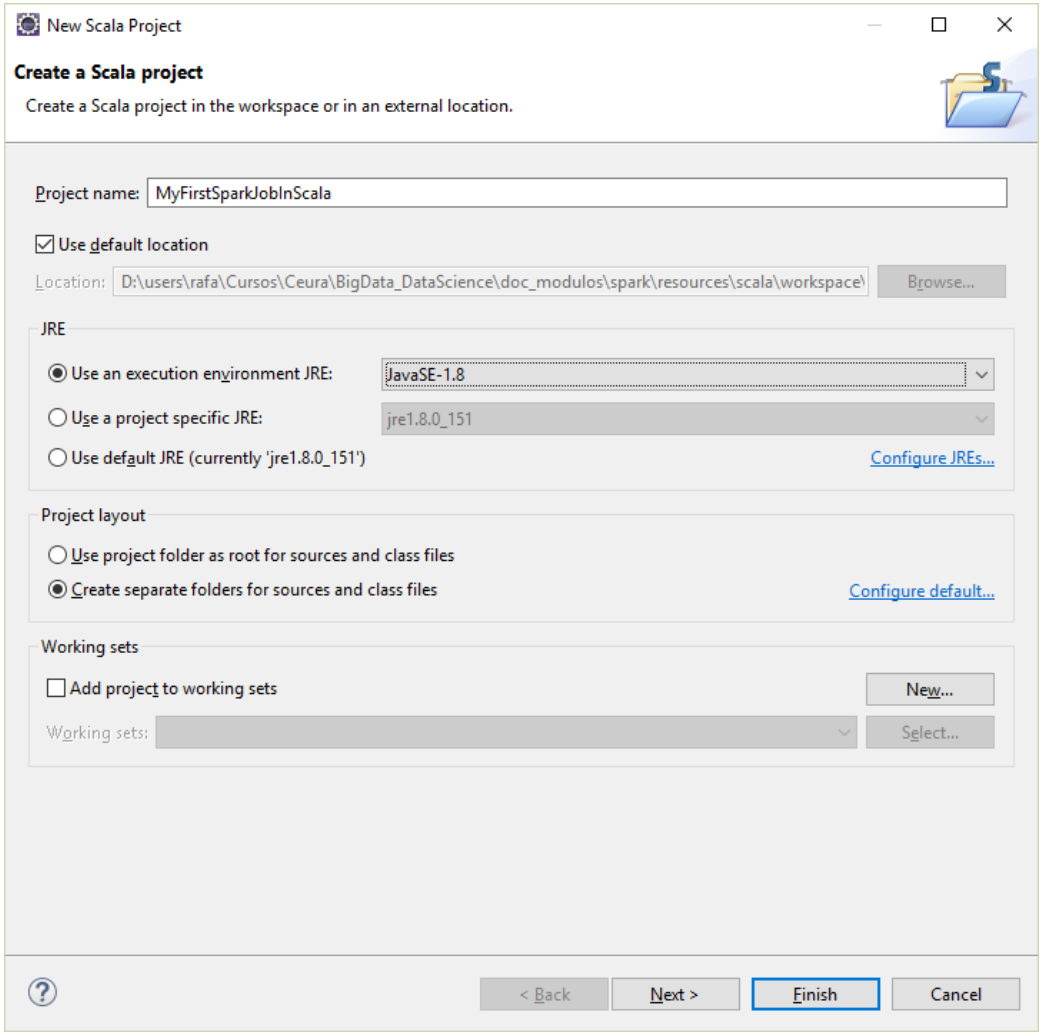
APLICACIONES/JOBS EN SCALA/JAVA (2/9)

- Instalar el plugin de Scala para Eclipse



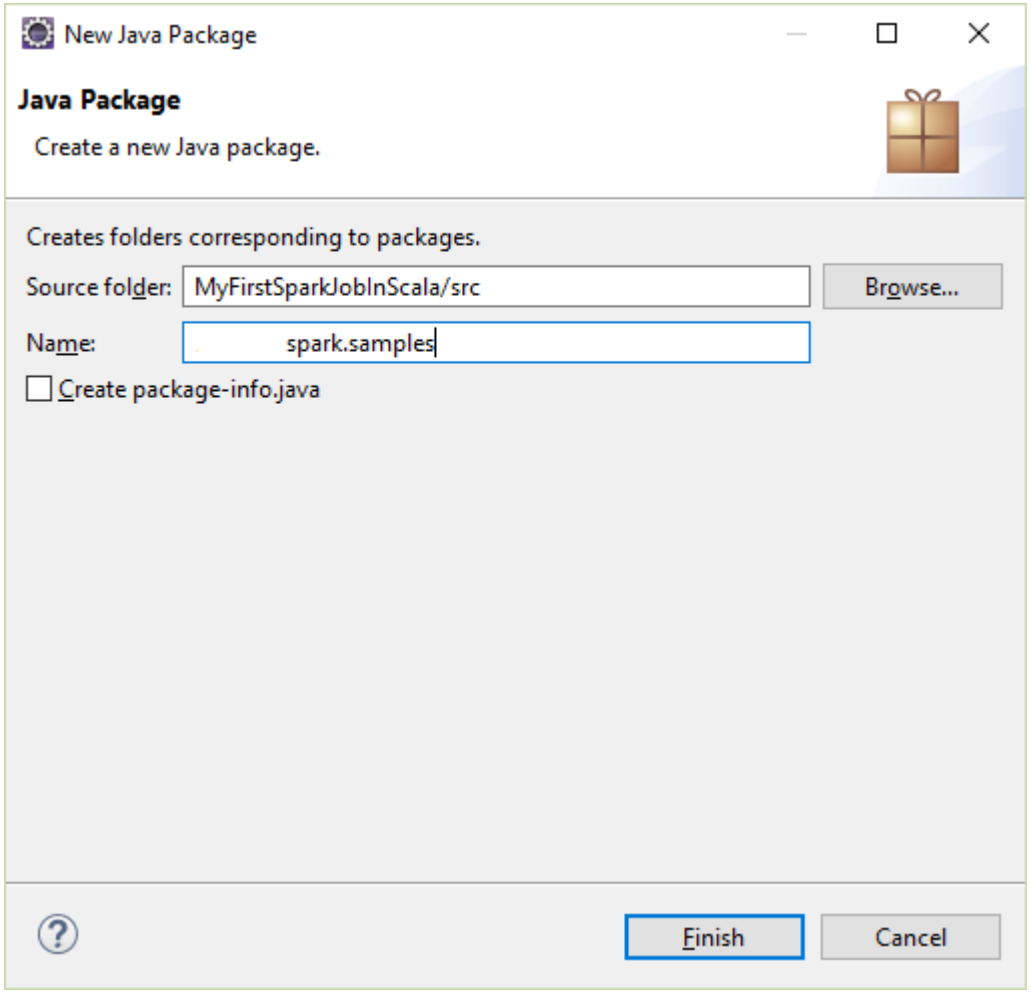
APLICACIONES/JOBS EN SCALA/JAVA (3/9)

- Crear un proyecto Scala
  - Usar los valores por defecto



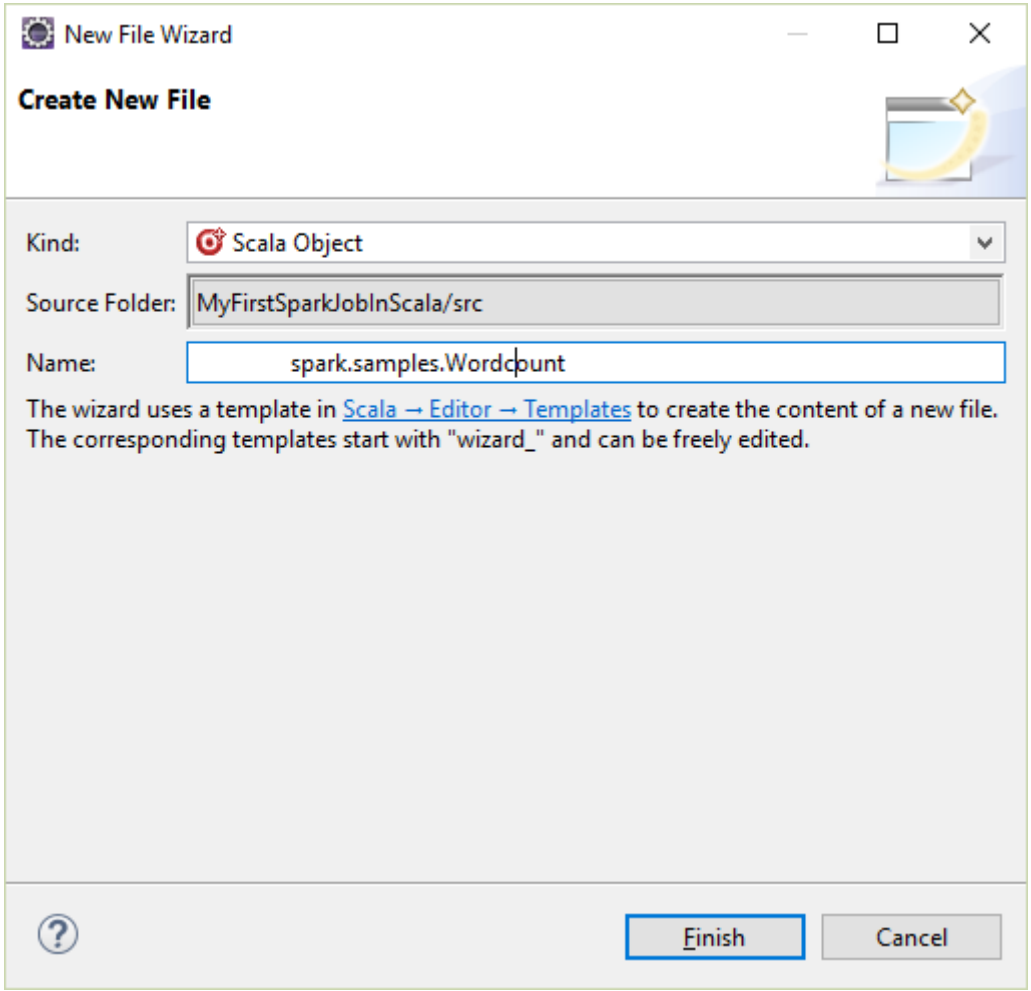
APLICACIONES/JOBS EN SCALA/JAVA (4/9)

- Crear un paquete



APLICACIONES/JOBS EN SCALA/JAVA (5/9)

- Crear un objeto Scala
  - Representa la aplicación/job
  - Ubicar dentro del paquete creado en el paso anterior



APLICACIONES/JOBS EN SCALA/JAVA (6/9)

Wordcount.scala

```scala
package spark.samples

import org.apache.spark.SparkContext
import org.apache.spark.SparkConf
import org.apache.spark.rdd.RDD.rddToPairRDDFunctions

object Wordcount {

  def main(args: Array[String]) {

    //Create conf object
      val conf = new SparkConf().setAppName("WordCount")

      //create spark context object
      val sc = new SparkContext(conf)

     //Check whether sufficient params are supplied
      if (args.length < 2) {
        println("Usage: WordCount <input> <output>")
        System.exit(1)
      }

      //Read file and create RDD
      val rawData = sc.textFile(args(0))

      //convert the lines into words using flatMap operation
      val words = rawData.flatMap(line => line.split(" "))

      //count the individual words using map and reduceByKey operation
      val wordCount = words.map(word => (word, 1)).reduceByKey(_ + _)

      //Save the result
      wordCount.saveAsTextFile(args(1))

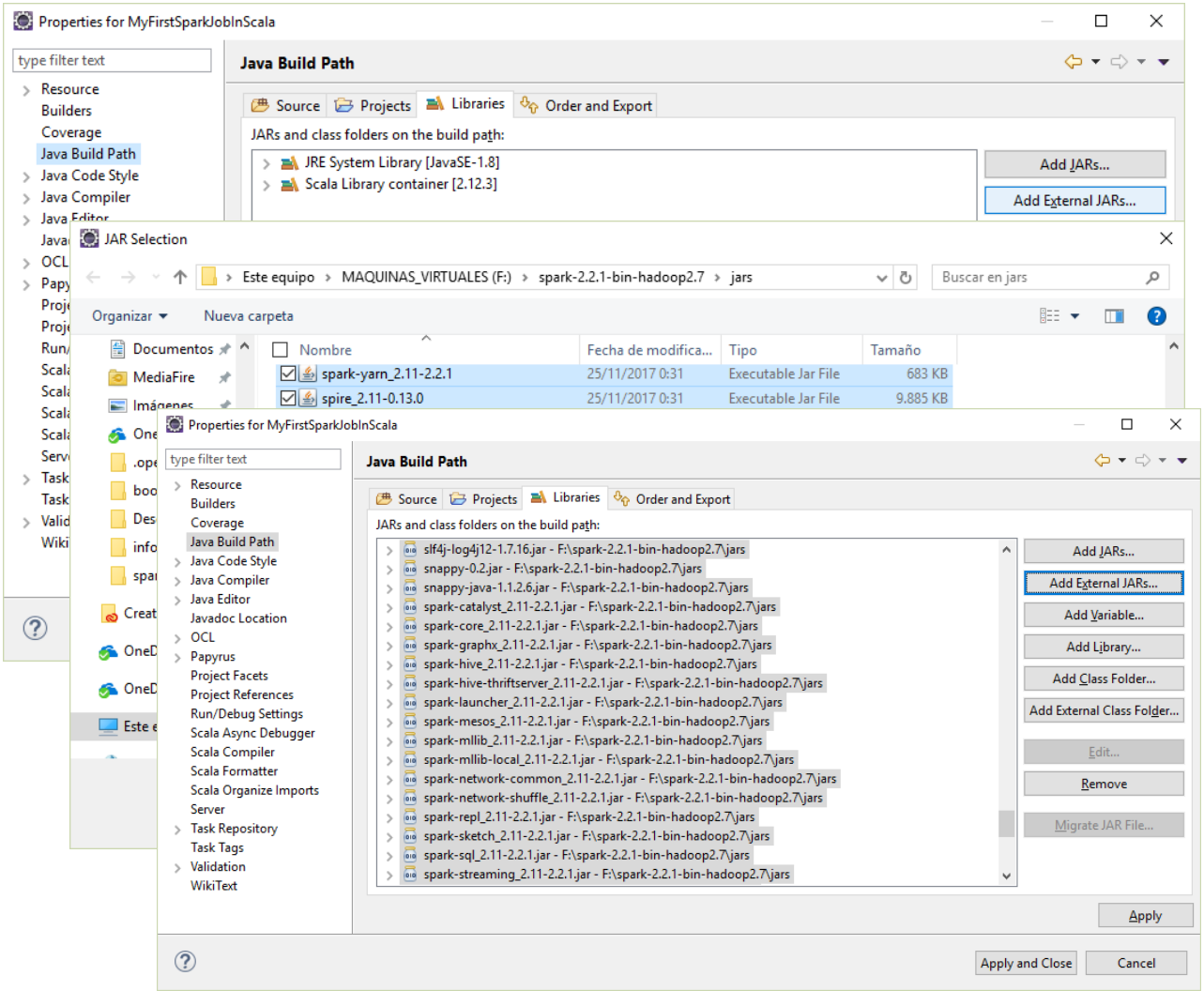
     //stop the spark context
      sc.stop
  }

}
```

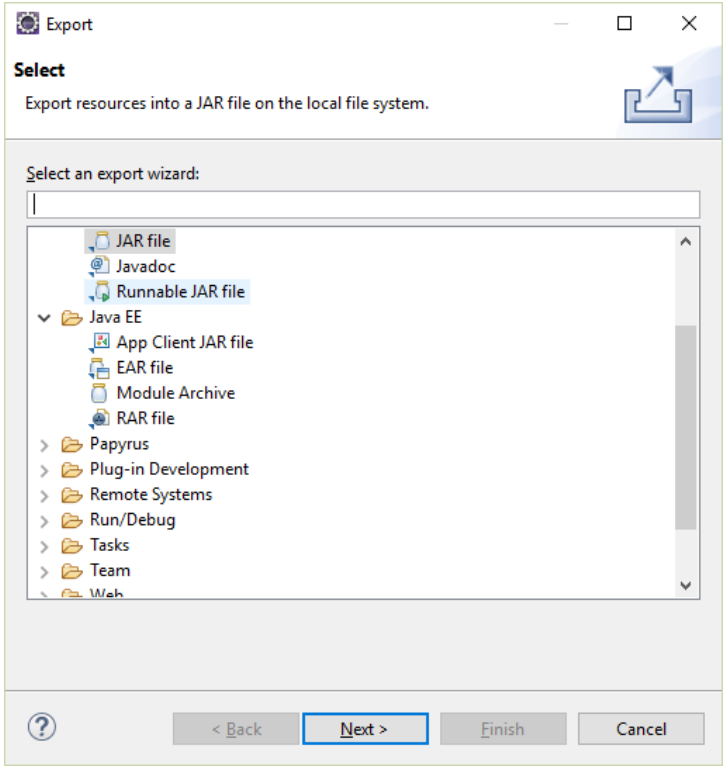


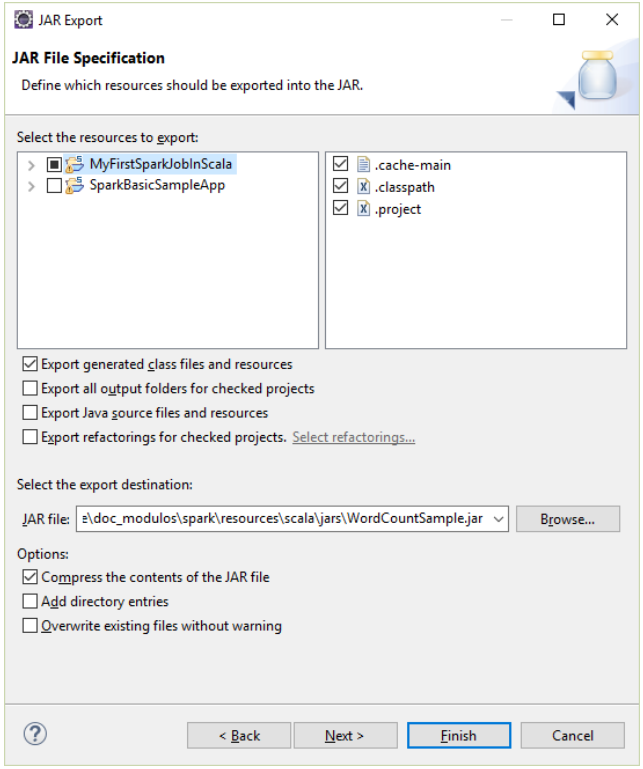
APLICACIONES/JOBS EN SCALA/JAVA (7/9)

- Agregar las librerias (ficheros jar) de Spark
  - Propiedades del proyecto → Java Build Path → Libraries
  - External JARS
    - TODOS!!!



APLICACIONES/JOBS EN SCALA/JAVA (8/9)





- Una vez resueltas las dependencias de compilación, se debe generar el fichero jar que representa a la aplicación/job
  - Export → JAR File

APLICACIONES/JOBS EN SCALA/JAVA (9/9)

- Ejecutar en spark

spark-submit --class **spark.samples.Wordcount** --master local ..\examples\jars**WordCountSample.jar** ..*Readme.md F:\spark_outputs\wordcount*

*spark-submit --class **spark.samples.Wordcount** --master local **file:///media/sf_data_spark/jars/WordCountSample.jar** file:///media/sf_data_spark/README.md file:///media/sf_data_spark/jars/output*

